ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

Description des agents et de leurs rôles

4ème année

IASDR Group 4

Sous le thème

Assistant d'Aide à l'Enseignement Primaire

(Éveil à la pensée informatique)

Réalisé par :

Lahbichi Alae

Benladoul Hyathame

Encadré par :

Chaabi Hasnaa

Année Universitaire :2025-2026

TABLE DES MATIÈRES

TABLE DES MATIÈRES ..... *II*

LISTE DES FIGURES *4*

INTRODUCTION GENERALE *5*

Rôle du RAG et les documents à utiliser ..... *Error! Bookmark not defined.*

1. Définition du RAG ..... Error! Bookmark not defined.

2. Rôle du RAG dans notre projet ..... Error! Bookmark not defined.

3. Objectifs de l'utilisation du RAG ..... Error! Bookmark not defined.

3.1 Améliorer la pertinence des activités générées ..... **Error! Bookmark not defined.**

3.2 Fournir un contexte pédagogique aux agents ..... **Error! Bookmark not defined.**

3.3 Adapter les activités au niveau des élèves ..... **Error! Bookmark not defined.**

3.4 Réduire les erreurs de génération ..... **Error! Bookmark not defined.**

3.5 Faciliter l'évolution du système ..... **Error! Bookmark not defined.**

4. Documents utilisés dans le RAG ..... Error! Bookmark not defined.

5. Exemples de contenus à intégrer dans la base documentaire ..... Error! Bookmark not defined.

6. Préparation des documents ..... Error! Bookmark not defined.

6.1 Chargement depuis Google Drive ..... **Error! Bookmark not defined.**

6.2 Extraction du texte ..... **Error! Bookmark not defined.**

6.3 Découpage en chunks ..... **Error! Bookmark not defined.**

6.4 Utilisation du découpage récursif ..... **Error! Bookmark not defined.**

6.5 Génération des embeddings ..... **Error! Bookmark not defined.**

6.6 Stockage dans ChromaDB ..... **Error! Bookmark not defined.**

7. Fonctionnement du RAG lors d'une requête utilisateur ..... Error! Bookmark not defined.

8. Exemple de fonctionnement du RAG ..... Error! Bookmark not defined.

9. Structure recommandée des documents ..... Error! Bookmark not defined.

10. Métadonnées utiles pour améliorer le RAG ..... Error! Bookmark not defined.

11. Avantages du RAG dans le projet ..... Error! Bookmark not defined.

12. Limites du RAG ..... Error! Bookmark not defined.

13. Améliorations possibles.....Error! Bookmark not defined.

14. Récapitulatif.....Error! Bookmark not defined.

CONCLUSION*.....*16

LISTE DES FIGURES

Figure 1 . Synthèse du chapitre de RAG..... **Error! Bookmark not defined.**

INTRODUCTION GENERALE

La quatrième semaine du projet est consacrée à la description des différents agents intelligents utilisés dans notre système. Après avoir défini les objectifs du projet, le workflow global et le rôle du RAG, cette partie permet d'expliquer comment les agents collaborent entre eux pour assurer la génération, l'organisation, l'évaluation et l'adaptation des activités pédagogiques.Le projet repose sur un Système Multi-Agents, c'est-à-dire une architecture composée de plusieurs agents spécialisés. Chaque agent possède une mission précise et intervient à une étape particulière du processus. Cette organisation permet de diviser une tâche complexe en plusieurs sous-tâches plus simples, mieux contrôlées et plus faciles à améliorer.Dans notre cas, les agents travaillent ensemble pour aider l'enseignant à créer des activités ludiques liées à l'éveil à la pensée informatique. Le système commence par recevoir la demande de l'utilisateur, puis génère une activité, l'organise sous une forme claire, analyse la réponse de l'élève et propose ensuite une nouvelle activité avec une difficulté adaptée.

Description des Agents et de leurs Rôles

1. Principe du Système Multi-Agents

Un **Système Multi-Agents**, ou SMA, est un système composé de plusieurs agents autonomes qui collaborent pour atteindre un objectif commun. Chaque agent est spécialisé dans une tâche précise et communique avec les autres agents afin de construire une solution complète.

Dans ce projet, l'objectif commun des agents est de produire un accompagnement pédagogique intelligent. Au lieu d'utiliser un seul agent qui fait tout, nous avons séparé les responsabilités afin d'obtenir un système plus clair, plus organisé et plus fiable.

Cette approche présente plusieurs avantages. Elle permet de mieux contrôler le rôle de chaque agent, d'améliorer facilement une partie du système sans modifier toute l'architecture, et de rendre le workflow plus compréhensible dans n8n.

2. Architecture générale des agents

Le système est organisé autour de plusieurs agents principaux :

Agent	Rôle principal
Agent de réception	Reçoit et interprète la requête de l'utilisateur.
Agent de génération	Crée une activité pédagogique adaptée.
Agent d'organisation	Structure l'activité pour l'affichage.
Agent d'évaluation	Analyse la réponse de l'utilisateur et donne un verdict.
Agent d'adaptation	Ajuste la difficulté et génère une nouvelle activité.
Agent RAG / Recherche documentaire	Récupère les informations pertinentes depuis ChromaDB.

Ces agents sont orchestrés par **n8n**, qui assure la circulation des données entre l'interface Flask, les agents IA, Ollama/Mistral, ChromaDB et les API REST.

3. Agent de réception de la requête utilisateur

3.1 Rôle de l'agent

L'agent de réception est le premier agent qui intervient dans le système. Son rôle est de recevoir la demande envoyée par l'utilisateur depuis l'interface Flask, puis de comprendre l'intention générale de cette demande. Par exemple, l'utilisateur peut demander :

Génère une activité sur les suites logiques pour des élèves du primaire.

L'agent doit identifier le thème principal, le type d'activité demandé et le niveau de difficulté souhaité, même si ces informations ne sont pas toujours formulées clairement.

3.2 Responsabilités principales

Cet agent a plusieurs responsabilités :

1. Récupérer la requête envoyée depuis Flask ;
2. Identifier le thème de l'activité ;
3. Détecter le besoin pédagogique ;
4. Préparer la demande pour les autres agents ;
5. Transmettre les informations nécessaires à l'agent de génération.

Il joue donc le rôle de point d'entrée du système.

3.3 Entrées et sorties de l'agent

Élément	Description
Entrée	Requête utilisateur envoyée depuis l'interface Flask.
Traitement	Analyse de la demande et extraction des informations importantes.
Sortie	Demande structurée transmise à l'agent de génération.

4. Agent RAG / Recherche documentaire

4.1 Rôle de l'agent RAG

L'agent RAG intervient pour enrichir la génération de l'activité. Son rôle est de rechercher dans la base vectorielle **ChromaDB** les passages les plus pertinents par rapport à la demande de l'utilisateur. Grâce à cet agent, le système ne dépend pas uniquement des connaissances générales du modèle Mistral. Il peut utiliser des documents pédagogiques déjà indexés pour produire des activités plus adaptées, plus fiables et plus proches du contexte scolaire.

4.2 Fonctionnement de l'agent RAG

Lorsque la demande utilisateur est reçue, elle est transformée en vecteur à l'aide du modèle d'embeddings **all-minilm:latest**. Ce vecteur est ensuite comparé aux vecteurs stockés dans **ChromaDB**. L'agent récupère les chunks les plus proches de la requête, puis les transmet à l'agent de génération sous forme de contexte.

4.3 Exemple d'utilisation

Si l'utilisateur demande une activité sur la logique algorithmique, l'agent RAG peut récupérer des passages liés aux séquences d'actions, à la résolution de problèmes ou aux instructions ordonnées. Ces informations permettent ensuite à l'agent de génération de produire une activité plus pertinente.

5. Agent de génération d'activité

5.1 Rôle de l'agent

L'agent de génération est l'un des agents les plus importants du système. Son rôle est de créer une activité pédagogique à partir de la demande de l'utilisateur et du contexte fourni par le RAG. Cet agent utilise le modèle **Mistral** exécuté localement avec **Ollama**. Il doit produire une activité adaptée aux élèves du primaire, en évitant les termes trop techniques et en privilégiant des situations simples, concrètes et ludiques.

5.2 Responsabilités principales

L'agent de génération doit :

- Comprendre le thème demandé ;
- Utiliser le contexte récupéré depuis ChromaDB ;
- Proposer une activité adaptée au niveau primaire ;
- Formuler un énoncé clair ;
- Créer une question compréhensible ;
- Proposer plusieurs choix de réponses ;
- Prévoir une logique de correction.

5.3 Exemple de sortie attendue

L'agent peut générer une activité de ce type :

Activité : Le chemin du petit robot

Un petit robot veut rejoindre son école. Pour cela, il doit suivre les instructions dans le bon ordre.

Question : Quelle suite d'actions permet au robot d'arriver à l'école ?

Tourner à gauche, avancer, reculer

Avancer, tourner à droite, avancer

Reculer, tourner à gauche, avancer

Tourner à droite, reculer, avancer

Cette première version peut ensuite être améliorée par l'agent d'organisation.

6. Agent d'organisation de l'activité

6.1 Rôle de l'agent

L'agent d'organisation intervient après la génération de l'activité. Son rôle est de transformer le contenu brut produit par l'agent de génération en une activité claire, structurée et prête à être affichée dans l'interface Flask. Même si l'agent de génération produit une activité correcte, le

contenu peut parfois manquer d'organisation ou contenir des formulations longues. L'agent d'organisation permet donc d'améliorer la présentation finale.

6.2 Responsabilités principales

L'agent d'organisation doit :

- Donner un titre clair à l'activité ;
- Séparer l'objectif pédagogique, l'énoncé, la question et les réponses ;
- Simplifier les phrases si nécessaire ;
- Rendre les consignes plus compréhensibles ;
- Vérifier que les choix de réponses sont bien numérotés ;
- Préparer le contenu pour l'affichage dans l'interface.

6.3 Structure finale proposée

L'activité organisée peut suivre cette structure :

```
Titre de l'activité :
Objectif pédagogique :
Énoncé :
Consigne :
Question :
Réponses possibles :
1.
2.
3.
4.
```

Cette structure rend l'activité plus lisible pour l'enseignant et plus facile à comprendre pour l'élève.

7. Agent d'évaluation de la réponse

7.1 Rôle de l'agent

L'agent d'évaluation intervient lorsque l'utilisateur répond à l'activité. Son rôle est d'analyser la réponse donnée, de vérifier si elle est correcte ou non, puis de produire un verdict pédagogique. Cet agent ne doit pas seulement dire si la réponse est vraie ou fausse. Il doit aussi fournir une explication simple, afin d'aider l'élève à comprendre son erreur ou à renforcer sa bonne réponse.

7.2 Responsabilités principales

L'agent d'évaluation doit :

- Recevoir la réponse de l'utilisateur ;
- Comparer la réponse avec la solution attendue ;
- Déterminer si la réponse est correcte, partiellement correcte ou incorrecte ;
- Produire un feedback clair ;
- Transmettre le résultat à l'agent d'adaptation.

7.3 Types de verdicts possibles

Verdict	Signification
Correct	L'utilisateur a choisi la bonne réponse.
Partiellement correct	La réponse contient une bonne idée, mais elle reste incomplète.
Incorrect	La réponse ne correspond pas à la solution attendue.

7.4 Exemple de feedback

Si l'élève répond correctement :

Bravo ! Ta réponse est correcte. Tu as bien compris l'ordre logique des actions.

Si l'élève répond incorrectement :

Ce n'est pas encore la bonne réponse. Observe bien l'ordre des étapes : il faut commencer par l'action qui permet au robot d'avancer vers son objectif.

Ce feedback garde un ton positif et encourageant.

8. Agent d'adaptation de la difficulté

8.1 Rôle de l'agent

L'agent d'adaptation est responsable de la personnalisation du parcours pédagogique. Après l'évaluation de la réponse, il décide si la prochaine activité doit être plus simple, équivalente ou plus difficile. Cet agent permet au système de proposer un apprentissage progressif. L'élève ne reçoit pas toujours des activités du même niveau : la difficulté évolue selon ses performances.

8.2 Logique d'adaptation

La logique d'adaptation peut être définie ainsi :

Résultat de l'élève	Décision du système
Réponse correcte	Augmenter légèrement la difficulté.
Réponse partiellement correcte	Garder une difficulté similaire avec plus d'aide.
Réponse incorrecte	Diminuer la difficulté ou proposer une activité plus guidée.

8.3 Exemple d'adaptation

Si l'élève réussit une activité simple sur l'ordre des actions, le système peut proposer ensuite une activité plus complexe avec plus d'étapes.

Si l'élève échoue, le système peut proposer une activité plus simple avec moins d'éléments ou avec une consigne plus guidée.

Par exemple :

activité facile : remettre 3 actions dans le bon ordre ;

activité moyenne : remettre 5 actions dans le bon ordre ;

activité difficile : choisir le bon chemin en respectant plusieurs conditions.

9. Communication entre les agents

Les agents communiquent entre eux à travers le workflow n8n. Chaque agent reçoit une entrée, effectue un traitement précis, puis transmet sa sortie à l'agent suivant.

Le cycle de communication peut être résumé ainsi :

```
Requête utilisateur
        ↓
Agent de réception
        ↓
Agent RAG / Recherche documentaire
        ↓
Agent de génération
        ↓
Agent d'organisation
        ↓
```

```
Affichage dans Flask
        ↓
Réponse utilisateur
        ↓
Agent d'évaluation
        ↓
Agent d'adaptation
        ↓
Nouvelle activité
```

Cette organisation rend le système clair et modulaire. Chaque agent peut être amélioré séparément sans modifier tout le workflow.

10. Exemple de scénario complet

Pour mieux comprendre le rôle des agents, voici un exemple de scénario complet.

L'enseignant saisit dans l'interface :

Je veux une activité sur les suites logiques.

L'agent de réception analyse la demande et identifie le thème : suites logiques. L'agent RAG recherche ensuite dans ChromaDB des passages liés aux suites, aux motifs et au raisonnement logique. L'agent de génération utilise ces informations pour créer une activité où l'élève doit compléter une suite de formes. L'agent d'organisation structure ensuite l'activité en séparant le titre, l'objectif, l'énoncé, la question et les réponses possibles. L'activité est affichée dans l'interface Flask. L'élève choisit une réponse. L'agent d'évaluation vérifie cette réponse et fournit un feedback. Si l'élève a bien répondu, l'agent d'adaptation demande une nouvelle activité plus difficile. Si l'élève s'est trompé, il propose une activité plus simple. Ainsi, tous les agents collaborent pour créer une expérience pédagogique progressive et personnalisée.

11. Avantages de l'approche multi-agents

L'utilisation d'un système multi-agents apporte plusieurs avantages au projet. Premièrement, elle permet de mieux organiser les responsabilités. Chaque agent possède un rôle précis, ce qui rend le système plus facile à comprendre et à maintenir. Deuxièmement, cette architecture améliore la qualité des résultats. Un agent spécialisé dans l'organisation peut reformuler et structurer l'activité mieux qu'un agent unique qui ferait tout en une seule étape. Troisièmement, le système devient plus évolutif. Il est possible d'ajouter de nouveaux agents plus tard, par exemple un agent de génération d'images, un agent de création de PDF ou un agent de suivi de progression des élèves. Enfin, cette approche permet une meilleure adaptation pédagogique, car l'agent d'adaptation ajuste la difficulté selon les réponses de l'utilisateur.

12. Limites et améliorations possibles

Même si l'approche multi-agents est efficace, elle présente certaines limites. La qualité du système dépend fortement de la qualité des prompts donnés aux agents. Si les instructions sont imprécises, les réponses peuvent manquer de cohérence ou ne pas respecter le format attendu. Une autre limite concerne l'évaluation automatique des réponses. Certaines réponses peuvent être ambiguës, surtout si l'élève écrit une phrase au lieu de choisir une réponse numérotée. L'agent d'évaluation doit donc être amélioré pour mieux gérer les réponses libres.

Comme amélioration future, il serait possible d'ajouter :

- Un agent de génération de fiches PDF ;
- Un agent de génération d'images pédagogiques ;
- Un agent de suivi de progression des élèves ;
- Un agent de recommandation d'activités ;
- Un agent de vérification pédagogique avant affichage.

Ces améliorations permettraient de rendre l'assistant plus complet et plus utile pour les enseignants.

13. Récapitulatif

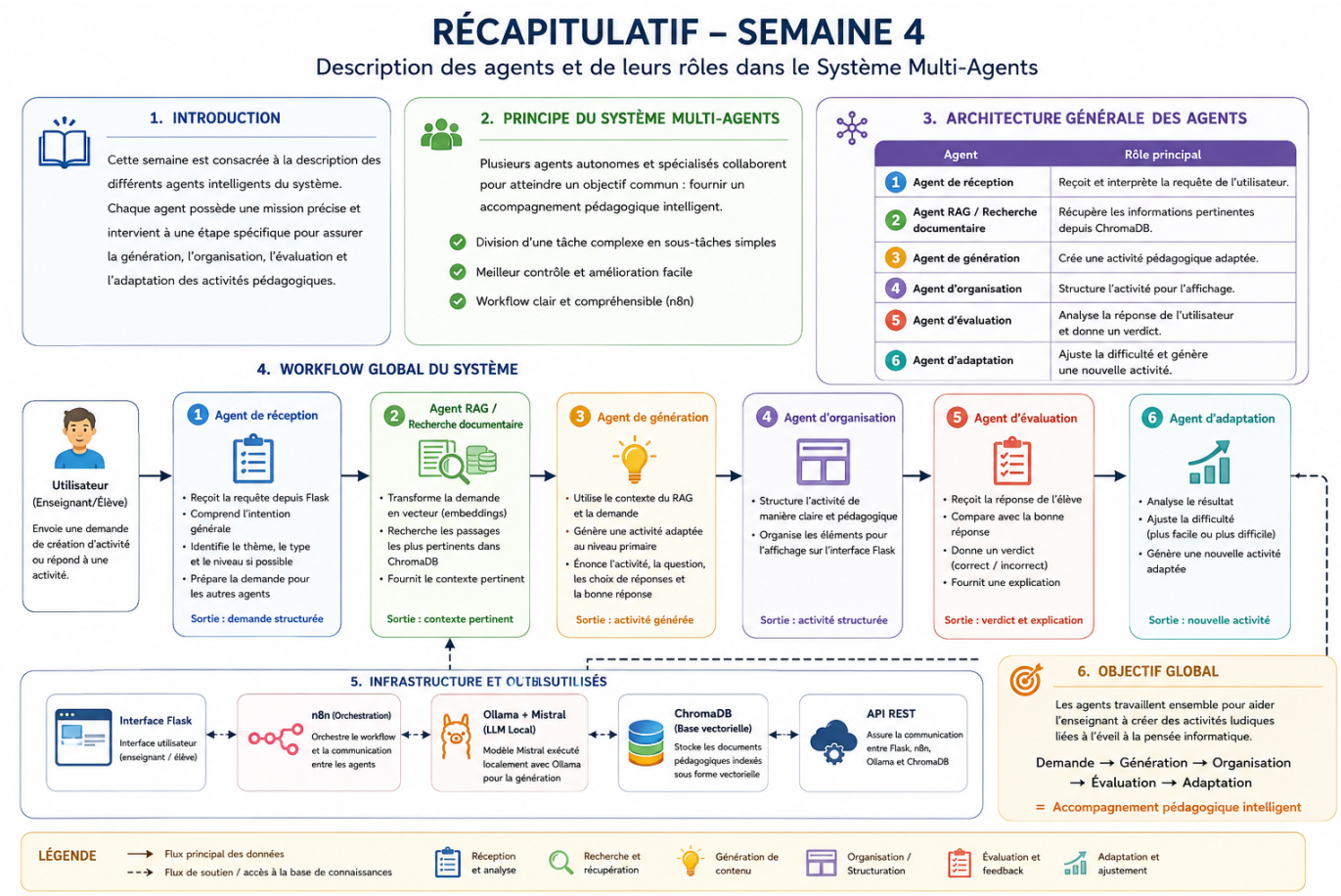

Figure 1 . Récapitulatif de la semaine 4

Cette figure présente une vue globale du fonctionnement du système multi-agents développé pour l'assistant pédagogique. Elle montre d'abord le principe général du système, où plusieurs agents spécialisés collaborent pour transformer une demande de l'utilisateur en activité pédagogique adaptée. Chaque agent possède un rôle précis : l'agent de réception analyse la requête, l'agent RAG recherche les informations pertinentes dans ChromaDB, l'agent de génération crée l'activité, l'agent d'organisation structure le contenu, l'agent d'évaluation analyse la réponse de l'élève, puis l'agent d'adaptation ajuste la difficulté pour proposer une nouvelle activité.La partie centrale de la figure illustre le workflow principal du système. Le processus commence par l'utilisateur, qui envoie une demande depuis l'interface Flask. Cette demande circule ensuite entre les différents agents selon un ordre logique : réception, recherche documentaire, génération, organisation, évaluation et adaptation. La figure met aussi en évidence la boucle d'amélioration continue, car après chaque réponse de l'élève, le système peut modifier le niveau de difficulté et générer une activité plus facile, similaire ou plus complexe.La partie inférieure présente les outils techniques utilisés dans le projet. Flask sert d'interface utilisateur, n8n assure l'orchestration du workflow, Ollama avec Mistral permet la génération de contenu, ChromaDB stocke les documents pédagogiques sous forme vectorielle, et les API REST assurent la communication entre les différents composants. Ainsi, la figure résume clairement l'architecture fonctionnelle du système et montre comment les agents collaborent pour offrir un accompagnement pédagogique intelligent, progressif et personnalisé (voir Figure 1).

CONCLUSION

Cette quatrième semaine a permis de décrire les différents agents qui composent le système et d'expliquer leurs rôles dans le fonctionnement global du projet. L'approche multi-agents permet de diviser le processus pédagogique en plusieurs étapes : réception de la demande, recherche documentaire, génération de l'activité, organisation du contenu, évaluation de la réponse et adaptation de la difficulté.Chaque agent possède une mission précise et contribue à rendre le système plus intelligent, plus clair et plus adapté aux besoins des enseignants et des élèves. L'orchestration avec **n8n** permet de coordonner ces agents, tandis que **Flask**, **Ollama/Mistral**, **ChromaDB** et **all-minilm:latest** assurent respectivement l'interface, la génération de contenu, la recherche vectorielle et la représentation sémantique des documents.Ainsi, cette semaine finalise la description fonctionnelle du système en montrant comment les agents collaborent pour produire un assistant pédagogique adaptatif. Elle prépare également les perspectives d'amélioration, notamment l'ajout de fiches PDF, d'illustrations pédagogiques et d'un suivi plus avancé de la progression des élèves.